# Pedidos

Pepe Calderón, Fernando Sola, Daniel Ayala, Inma Hernández, Margarita Cruz, Carlos Arévalo, David Ruiz

## Contenido

1. Requisitos . . . 2
2. Requisitos de información (RI) . . . 2
   2.1. RI-1: Usuarios . . . 2
   2.2. RI-2: Productos . . . 2
   2.3. RI-3: Pedidos . . . 2
3. Reglas de negocio (RN) . . . 2
   3.1. RN-1: Límite de pedidos . . . 2
   3.2. RN-2: Agosto inhábil . . . 2
   3.3. RN-3: Stock suficiente . . . 2
4. Pruebas de aceptación (PA) . . . 3
   4.1. PA-1: Pedidos . . . 3
5. Modelo Conceptual . . . 3
6. Diagrama de clases . . . 3
7. Diagrama de objetos . . . 3
8. Modelo Relacional . . . 4
9. Intensión . . . 4
10. Extensión . . . 4
11. Álgebra relacional . . . 5
12. Modelo Tecnológico (MariaDB) . . . 5
13. Script SQL para crear la base de datos . . . 6
14. Script SQL para la carga inicial de datos . . . 7
15. Consultas . . . 8
16. Procedimientos . . . 13
17. Tests . . . 16

Versión PDF disponible

Pedidos

# 1. Requisitos

Catálogo de Requisitos

# 2. Requisitos de información (RI)

## 2.1. RI-1: Usuarios

- Como: Profesor de la asignatura
- Quiero: Poder almacenar la siguiente información de los usuarios: nombre, provincia y fecha de alta en el Sistema de Pedidos.
- Para: Que el estudiante realice a partir de este requisito el modelo conceptual, relacional y tecnológico

## 2.2. RI-2: Productos

- Como: Profesor de la asignatura
- Quiero: Poder almacenar la siguiente información de los productos: descripción, precio y stock en el almacén.
- Para: Que el estudiante realice a partir de este requisito el modelo conceptual, relacional y tecnológico

## 2.3. RI-3: Pedidos

- Como: Profesor de la asignatura
- Quiero: Poder almacenar la siguiente información de los pedidos: fecha de compra en la que el usuario hace el pedido y cantidad de productos que se piden.
- Para: Que el estudiante realice a partir de este requisito el modelo conceptual, relacional y tecnológico

# 3. Reglas de negocio (RN)

## 3.1. RN-1: Límite de pedidos

- Como: Profesor de la asignatura
- Quiero: Que un usuario no pueda hacer más de tres pedidos al día
- Para: Que el estudiante practique con restricciones simples

## 3.2. RN-2: Agosto inhábil

- Como: Profesor de la asignatura
- Quiero: Que no se puedan hacer pedidos en el mes de agosto
- Para: Que el estudiante practique con restricciones simples

## 3.3. RN-3: Stock suficiente

- Como: Profesor de la asignatura
- Quiero: Que solo se pueda realizar el pedido si hay stock suficiente
- Para: Que el estudiante practique con restricciones simples

# 4. Pruebas de aceptación (PA)

## 4.1. PA-1: Pedidos

1. ✅ Crear un nuevo pedido con todos los datos correctos según las reglas de negocio.
2. ❌ Crear un nuevo pedido sin especificar la cantidad de productos.
3. ❌ Crear un nuevo pedido sin especificar la fecha de compra.
4. ❌ Crear un nuevo pedido con más de tres pedidos al día.
5. ❌ Crear un nuevo pedido en el mes de agosto.
6. ❌ Crear un nuevo pedido sin stock suficiente.

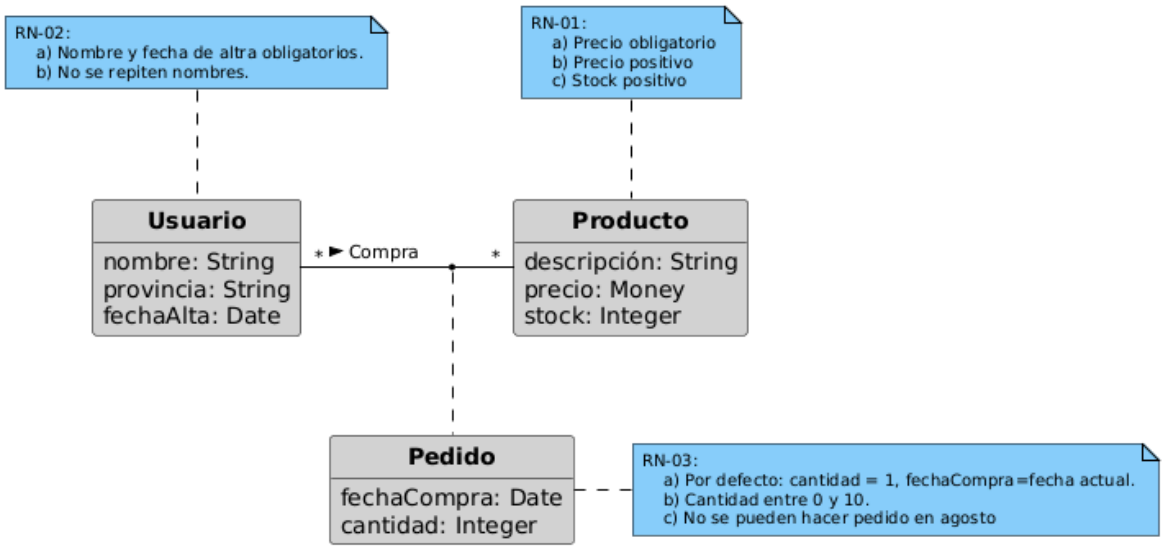
# 5. Modelo Conceptual

Modelo conceptual

# 6. Diagrama de clases

El modelo extiende el dominio de Usuarios incorporando las entidades Producto y Pedido para gestionar un sistema de comercio básico.

- **Entidades principales**: Usuario, Producto, Pedido.
- **asociaciones**: Un Usuario puede realizar 0..* Pedidos; un Pedido pertenece a 1 Usuario. Un Pedido incluye 1..* Productos; un Producto puede estar en 0..* Pedidos.
- **Restricciones de negocio**: Stock suficiente antes de crear pedido; máximo 3 pedidos por día por usuario; no pedidos en agosto.

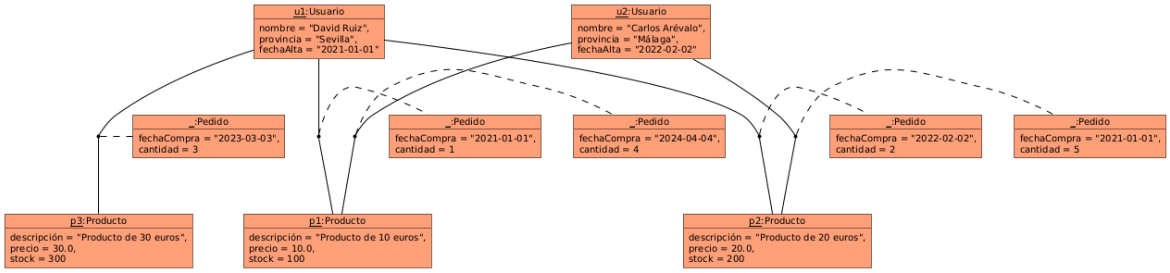



# 7. Diagrama de objetos

Los objetos ejemplifican escenarios típicos de uso del sistema de pedidos con diferentes patrones de compra.

- **Instancias de usuarios**: Varios usuarios (u1,u2) con pedidos realizados en fechas diferentes.
- **Productos**: Artículos con stock y precio variable (p1,p2,p3).
- **Pedidos**: Diferentes cantidades, fechas válidas (no agosto), respetando límite diario (se utiliza la notación para objetos anónimos _).

- **Vínculos**: Conexiones entre usuarios-pedidos y pedidos-productos mostrando las cardinalidades del modelo.




# 8. Modelo Relacional

Modelo relacional

# 9. Intensión

```
Usuarios(usuarioId, nombre, provincia, fechaAlta)
  PK(usuarioId)
Productos(productoId, descripcion, precio, stock)
  PK(productoId)
Pedidos(pedidoId, usuarioId, productoId, fechaCompra, cantidad)
  PK(pedidoId)
  FK(usuarioId) / Usuarios
  FK(productoId) / Productos
```

# 10. Extensión

```
Usuarios = {
  (u1, "David Ruiz", "Sevilla", "2018-05-18"),
  (u2, "Marta López", "Málaga", "2018-06-12"),
  (u3, "Raquel Lobato", "Granada", "2018-12-01"),
  (u4, "Antonio Gómez", "Sevilla", "2018-03-11"),
  (u5, "Inma Hernández", "Málaga", "2018-04-12"),
  (u6, "Jimena Martín", "Granada", "2018-05-13"),
  (u7, "Carlos Rivero", "Huelva", "2018-09-07"),
  (u8, "Carlos Arévalo", "Málaga", "2018-09-07")
}
Productos = {
  (p1, "Mi Band 3", 19.90, 50),
  (p2, "Mi Band 4", 29.90, 20),
  (p3, "Pulsera compatible con Mi Band 3 y 4", 9.90, 150),
  (p4, "Mi Scooter", 349.90, 25),
  (p5, "Rueda trasera de respuesto Mi Scooter", 19.90, 50),
  (p6, "Rueda delantera de respuesto Mi Scooter", 59.90, 50)
}
Pedidos = {
  (pe1, u1, p1, "2019-05-13", 2),
  (pe2, u1, p3, "2019-05-13", 2),
  (pe3, u2, p2, "2019-06-11", 3),
  (pe4, u2, p3, "2019-06-11", 1),
  (pe5, u3, p4, "2019-06-15", 2),
  (pe6, u4, p5, "2019-06-18", 1),
  (pe7, u4, p6, "2019-06-18", 1),
  (pe8, u5, p4, "2019-12-15", 2),
  (pe9, u7, p1, "2019-12-15", 1),
```

```
  (pe10, u7, p2, "2019-12-16", 1),
  (pe11, u7, p3, "2019-12-17", 1),
  (pe12, u7, p4, "2019-12-18", 1),
  (pe13, u7, p5, "2019-12-19", 1),
  (pe14, u7, p6, "2019-12-20", 1),
  (pe15, u8, p1, "2019-12-15", 1)
}
```

# 11. Álgebra relacional

Renombrado:

$$U(uId, n, p, fa) \leftarrow Usuarios(usuarioId, nombre, provincia, fechaAlta)$$

$$P(pId, d, p, s) \leftarrow Productos(productoId, descripcion, precio, stock)$$

$$Ped(pedId, uId, pId, fc, c) \leftarrow Pedidos(pedidoId, usuarioId, productoId, fechaCompra, cantidad)$$

Listados y consultas:

- Todos los pedidos con sus usuarios y productos:

$$UPP \leftarrow U \bowtie Ped \bowtie P$$

- Pedidos de usuarios de Málaga:

$$\underset{U.p=\text{ Málaga}}{\sigma}(UPP)$$

- Descripción y stock de productos con stock < 100:

$$\underset{P.d,P.s}{\Pi}(\underset{P.s<100}{\sigma}(P))$$

- Número de pedidos por usuario:

$$\overset{\text{COUNT}(pedId)}{\gamma} uId(U \bowtie Ped)$$

- Importe total por usuario:

$$\overset{\text{SUM}(P.p \cdot Ped.c)}{\gamma} uId(P \bowtie Ped)$$

- Pedidos por mes:

$$\overset{\text{COUNT}(pedId)}{\gamma} \text{MES}(Ped.fc)(Ped)$$

- Usuario que más gasta cada mes:

$$\overset{U.n,\ \text{MAX}(P.p \cdot Ped.c)}{\gamma} \text{MES}(Ped.fc)(U \bowtie P \bowtie Ped)$$

- Mes de máxima recaudación:

$$\overset{\text{MAX}(P.p \cdot Ped.c)}{\gamma}(P \bowtie Ped)$$

# 12. Modelo Tecnológico (MariaDB)

Modelo tecnológico (MariaDB)

# 13. Script SQL para crear la base de datos

**Pedidos/createDB.sql**

```sql
--
-- Autores: Inma Hernández, David Ruiz, y Daniel Ayala
-- Fecha creación: Noviembre de 2019
-- Fecha modificación: Noviembre 2020 (Carlos Arévalo)
--                     Noviembre 2025 (David Ruiz)
-- Descripción: Script para crear la BD del ejercicio de Pedidos
--

DROP DATABASE IF EXISTS PedidosDB;
CREATE DATABASE PedidosDB;
USE PedidosDB;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_create_db()
BEGIN
  -- Borrar tablas en orden correcto (FK primero)
  DROP TABLE IF EXISTS orders;
  DROP TABLE IF EXISTS products;
  DROP TABLE IF EXISTS users;

  -- Relación: Usuarios
  CREATE TABLE users(
    user_id INT NOT NULL AUTO_INCREMENT,
    name VARCHAR(63) NOT NULL,
    province VARCHAR(63),
    start_date DATE NOT NULL,
    PRIMARY KEY(user_id),
    CONSTRAINT RN_1a_name_not_null CHECK (name IS NOT NULL), -- RN-1a
    CONSTRAINT RN_1b_name_unique UNIQUE(name) -- RN-1b
  );

  -- Relación: Productos
  CREATE TABLE products(
    product_id INT NOT NULL AUTO_INCREMENT,
    description VARCHAR(128),
    price DECIMAL(6,2) NOT NULL,
    stock INT,
    PRIMARY KEY(product_id),
    CONSTRAINT RN_2a_price_not_null CHECK (price IS NOT NULL), -- RN-2a
    CONSTRAINT RN_2b_price_range CHECK (price >= 0), -- RN-2b
    CONSTRAINT RN_2c_positive_stock CHECK (stock >= 0) -- RN-2c
  );

  -- Relación: Pedidos
  CREATE TABLE orders(
    order_id INT NOT NULL AUTO_INCREMENT,
    user_id INT NOT NULL,
    product_id INT NOT NULL,
    amount INT DEFAULT 1,
    purchase_date DATE DEFAULT (CURRENT_DATE),
    PRIMARY KEY(order_id),
    FOREIGN KEY(user_id) REFERENCES users(user_id),
    FOREIGN KEY(product_id) REFERENCES products(product_id),
```

```
    CONSTRAINT RN_3a_amount_default CHECK (amount IS NOT NULL), -- RN-3a
    CONSTRAINT RN_3b_amount_range CHECK (amount BETWEEN 1 AND 10), -- RN-3b
    CONSTRAINT RN_3c_month_not_august CHECK (MONTH(purchase_date) <> 8) -- RN-3c
  );
END //
DELIMITER ;

CALL p_create_db();
```

# 14. Script SQL para la carga inicial de datos

**Pedidos/populateDB.sql**

```
--
-- Autores: Inma Hernández, David Ruiz, y Daniel Ayala
-- Fecha creación: Noviembre de 2019
-- Fecha modificación: Noviembre 2020 (Carlos Arévalo)
--                     Noviembre 2025 (David Ruiz)
-- Descripción: Script para hacer la carga inicial del ejercicio de Pedidos
--

USE PedidosDB;
-- Procedimiento para cargar datos iniciales (útil para tests)
DELIMITER //
CREATE OR REPLACE PROCEDURE p_populate_db()
BEGIN
  DELETE FROM orders;
  DELETE FROM products;
  DELETE FROM users;

  -- Datos de Usuarios
  INSERT INTO users(name, province, start_date)
  VALUES
    ('David Ruiz', 'Sevilla', '2018-05-18'),
    ('Marta López', 'Málaga', '2018-06-12'),
    ('Raquel Lobato', 'Granada', '2018-12-01'),
    ('Antonio Gómez', 'Sevilla', '2018-03-11'),
    ('Inma Hernández', 'Málaga', '2018-04-12'),
    ('Jimena Martín', 'Granada', '2018-05-13'),
    ('Carlos Rivero', 'Huelva', '2018-09-07'),
    ('Carlos Arévalo', 'Málaga', '2018-09-07');

  -- Datos de Productos
  INSERT INTO products(description, price, stock)
  VALUES
    ('Mi Band 3', 19.90, 50),
    ('Mi Band 4', 29.90, 20),
    ('Pulsera compatible con Mi Band 3 y 4', 9.90, 150),
    ('Mi Scooter', 349.90, 25),
    ('Rueda trasera de respuesto Mi Scooter', 19.90, 50),
    ('Rueda delantera de respuesto Mi Scooter', 59.90, 50);

  -- Datos de Pedidos
  INSERT INTO orders(user_id, product_id, amount, purchase_date)
  VALUES
```

```
    (1, 1, 2, '2019-05-13'),
    (1, 3, 2, '2019-05-13'),
    (2, 2, 3, '2019-06-11'),
    (2, 3, 1, '2019-06-11'),
    (3, 4, 2, '2019-06-15'),
    (4, 5, 1, '2019-06-18'),
    (4, 6, 1, '2019-06-18'),
    (5, 4, 2, '2019-12-15'),
    (7, 1, 1, '2019-12-15'),
    (7, 2, 1, '2019-12-16'),
    (7, 3, 1, '2019-12-17'),
    (7, 4, 1, '2019-12-18'),
    (7, 5, 1, '2019-12-19'),
    (7, 6, 1, '2019-12-20'),
    (8, 1, 1, '2019-12-15');
END //
DELIMITER ;

CALL p_populate_db();
```

# 15. Consultas

## Pedidos/queries.sql

```
--
-- Autor: Carlos Arévalo
-- Fecha creación: Noviembre de 2020
-- Fecha modificación: Noviembre 2025 (David Ruiz)
-- Descripción: Consultas SQL de ejemplo para el ejercicio de Pedidos
--

USE PedidosDB;

-- 1. Usuarios de Sevilla
SELECT *
FROM users
WHERE province = 'Sevilla';

-- 2. Usuarios que no son de Sevilla
SELECT *
FROM users
WHERE province <> 'Sevilla';

-- 3. Malagueños y sevillanos
SELECT name
FROM users
WHERE province IN ('Sevilla', 'Málaga');

-- 4. Nombre de los usuarios, descripción del producto, cantidad solicitada y
total en euros del pedido
SELECT name, description, amount, amount * price AS total_order
FROM users
NATURAL JOIN orders
NATURAL JOIN products;
```

```sql
-- 5. Usuarios sin Pedidos
-- (a) Opción 1
SELECT name, user_id
FROM users
EXCEPT
SELECT name, user_id
FROM users
NATURAL JOIN orders;

-- (b) Opción 2
SELECT name, user_id
FROM users U
WHERE NOT EXISTS (
  SELECT *
  FROM orders O
  WHERE O.user_id = U.user_id
);

-- 6. Usuarios que han comprado todos los productos
-- (a) Como doble diferencia, según la equivalencia algebráica de teoría
SELECT name
FROM users
NATURAL JOIN (
  SELECT user_id
  FROM orders
  EXCEPT
  SELECT X.user_id
  FROM (
    SELECT O.user_id, P.product_id
    FROM orders O, products P
    EXCEPT
    SELECT user_id, product_id
    FROM orders
    NATURAL JOIN products
  ) X
) Y;

-- (b) Con NOT EXISTS
SELECT name
FROM users U
WHERE NOT EXISTS (
  SELECT *
  FROM products P
  WHERE NOT EXISTS (
    SELECT *
    FROM orders O
    WHERE O.product_id = P.product_id
    AND O.user_id = U.user_id
  )
);

-- 7. Nombre de usuario con sus pedidos
SELECT name, SUM(amount * price) AS money_orders
FROM orders
NATURAL JOIN users
NATURAL JOIN products
```

```sql
GROUP BY name
ORDER BY 2 DESC;

-- 8. Producto con su número de pedidos
SELECT description, COUNT(order_id) AS amount_orders
FROM orders
NATURAL JOIN users
NATURAL JOIN products
GROUP BY description
ORDER BY 2 DESC;

-- 9. Creación de vistas y consultas sobre ellas
-- (a) Resumen de pedidos por provincia (totales, media, desviación típica,
máximo, minimo)
CREATE OR REPLACE VIEW v_province_stat AS
SELECT province,
  COUNT(*) AS num_orders,
  SUM(amount) AS sum_amount,
  MAX(amount) AS max_amount,
  MIN(amount) AS min_amount,
  AVG(amount) AS avg_amount,
  STD(amount) AS std_amount,
  SUM(price * amount) AS sum_money_orders,
  MAX(price * amount) AS max_money_orders,
  MIN(price * amount) AS min_money_orders,
  AVG(price * amount) AS avg_money_orders,
  STD(price * amount) AS std_money_orders
FROM orders
NATURAL JOIN products
NATURAL JOIN users
GROUP BY province
ORDER BY province ASC;

-- (b) Resumen de pedidos por producto (totales, medias, máximo, minimo)
CREATE OR REPLACE VIEW v_product_stat AS
SELECT description,
  COUNT(*) AS num_orders,
  SUM(amount) AS sum_amount,
  MAX(amount) AS max_amount,
  MIN(amount) AS min_amount,
  AVG(amount) AS avg_amount,
  STD(amount) AS std_amount,
  SUM(price * amount) AS sum_money_orders,
  MAX(price * amount) AS max_money_orders,
  MIN(price * amount) AS min_money_orders,
  AVG(price * amount) AS avg_money_orders,
  STD(price * amount) AS std_money_orders
FROM orders
NATURAL JOIN products
GROUP BY description
ORDER BY description ASC;

-- (c) Resumen de pedidos por usuarios (totales, medias, máximo, minimo)
CREATE OR REPLACE VIEW v_user_stat AS
SELECT name,
  COUNT(*) AS num_orders,
```

```sql
  SUM(amount) AS sum_amount,
  MAX(amount) AS max_amount,
  MIN(amount) AS min_amount,
  AVG(amount) AS avg_amount,
  STD(amount) AS std_amount,
  SUM(price * amount) AS sum_money_orders,
  MAX(price * amount) AS max_money_orders,
  MIN(price * amount) AS min_money_orders,
  AVG(price * amount) AS avg_money_orders,
  STD(price * amount) AS std_money_orders
FROM orders
NATURAL JOIN users
NATURAL JOIN products
GROUP BY name
ORDER BY name ASC;

-- (d) Usuario que ha comprado más y menos
-- Opción 1 (MAX, MIN)
SELECT name, sum_money_orders
FROM v_user_stat
WHERE sum_money_orders = (SELECT MAX(sum_money_orders) FROM v_user_stat);

SELECT name, sum_money_orders
FROM v_user_stat
WHERE sum_money_orders = (SELECT MIN(sum_money_orders) FROM v_user_stat);

-- Opción 2 (ALL)
SELECT name, sum_money_orders
FROM v_user_stat
WHERE sum_money_orders >= ALL (SELECT sum_money_orders FROM v_user_stat);

SELECT name, sum_money_orders
FROM v_user_stat
WHERE sum_money_orders <= ALL (SELECT sum_money_orders FROM v_user_stat);

-- (e) Pedido de mayor y menor cuantía
-- Opción 1 (MAX, MIN)
SELECT order_id, user_id, amount * price AS sum_money_orders
FROM orders
NATURAL JOIN products
WHERE amount * price = (SELECT MAX(amount * price) FROM orders NATURAL JOIN
products);

SELECT order_id, user_id, amount * price AS sum_money_orders
FROM orders
NATURAL JOIN products
WHERE amount * price = (SELECT MIN(amount * price) FROM orders NATURAL JOIN
products);

-- Opción 2 (ALL)
SELECT order_id, user_id, amount * price AS sum_money_orders
FROM orders
NATURAL JOIN products
WHERE amount * price >= ALL (SELECT amount * price FROM orders NATURAL JOIN
products);
```

```sql
SELECT order_id, user_id, amount * price AS sum_money_orders
FROM orders
NATURAL JOIN products
WHERE amount * price <= ALL (SELECT amount * price FROM orders NATURAL JOIN
products);

-- (b) Resumen de pedidos por producto (totales, medias, máximo, minimo)
CREATE OR REPLACE VIEW V_Product_Stat AS
SELECT description,
  COUNT(*) AS numOrders,
  SUM(amount) AS sumAmount,
  MAX(amount) AS maxAmount,
  MIN(amount) AS minAmount,
  AVG(amount) AS avgAmount,
  STD(amount) AS stdAmount,
  SUM(price * amount) AS sumMoneyOrders,
  MAX(price * amount) AS maxMoneyOrders,
  MIN(price * amount) AS minMoneyOrders,
  AVG(price * amount) AS avgMoneyOrders,
  STD(price * amount) AS stdMoneyOrders
FROM Orders
NATURAL JOIN Products
GROUP BY description
ORDER BY description ASC;

-- (c) Resumen de pedidos por usuarios (totales, medias, máximo, minimo)
CREATE OR REPLACE VIEW V_User_Stat AS
SELECT name,
  COUNT(*) AS numOrders,
  SUM(amount) AS sumAmount,
  MAX(amount) AS maxAmount,
  MIN(amount) AS minAmount,
  AVG(amount) AS avgAmount,
  STD(amount) AS stdAmount,
  SUM(price * amount) AS sumMoneyOrders,
  MAX(price * amount) AS maxMoneyOrders,
  MIN(price * amount) AS minMoneyOrders,
  AVG(price * amount) AS avgMoneyOrders,
  STD(price * amount) AS stdMoneyOrders
FROM Orders
NATURAL JOIN Users
NATURAL JOIN Products
GROUP BY name
ORDER BY name ASC;

-- (d) Usuario que ha comprado más y menos
-- Opción 1 (MAX, MIN)
SELECT name, sumMoneyOrders
FROM V_User_Stat
WHERE sumMoneyOrders = (SELECT MAX(sumMoneyOrders) FROM V_User_Stat);

SELECT name, sumMoneyOrders
FROM V_User_Stat
WHERE sumMoneyOrders = (SELECT MIN(sumMoneyOrders) FROM V_User_Stat);

-- Opción 2 (ALL)
```

```sql
SELECT name, sumMoneyOrders
FROM V_User_Stat
WHERE sumMoneyOrders >= ALL (SELECT sumMoneyOrders FROM V_User_Stat);

SELECT name, sumMoneyOrders
FROM V_User_Stat
WHERE sumMoneyOrders <= ALL (SELECT sumMoneyOrders FROM V_User_Stat);

-- (e) Pedido de mayor y menor cuantía
-- Opción 1 (MAX, MIN)
SELECT orderId, userId, amount * price AS sumMoneyOrders
FROM Orders
NATURAL JOIN Products
WHERE amount * price = (SELECT MAX(amount * price) FROM Orders NATURAL JOIN
Products);

SELECT orderId, userId, amount * price AS sumMoneyOrders
FROM Orders
NATURAL JOIN Products
WHERE amount * price = (SELECT MIN(amount * price) FROM Orders NATURAL JOIN
Products);

-- Opción 2 (ALL)
SELECT orderId, userId, amount * price AS sumMoneyOrders
FROM Orders
NATURAL JOIN Products
WHERE amount * price >= ALL (SELECT amount * price FROM Orders NATURAL JOIN
Products);

SELECT orderId, userId, amount * price AS sumMoneyOrders
FROM Orders
NATURAL JOIN Products
WHERE amount * price <= ALL (SELECT amount * price FROM Orders NATURAL JOIN
Products);
```

# 16. Procedimientos

## Pedidos/procedures.sql

```sql
--
-- Autores: Daniel Ayala, Inma Hernández y David Ruiz
-- Fecha creación: Noviembre de 2019
-- Fecha modificación: Noviembre 2020 (Carlos Arévalo)
--                     Noviembre 2025 (David Ruiz)
-- Descripción: Procedimientos, funciones y disparadores para el ejercicio de
Pedidos
--

USE PedidosDB;

-- Función que dado el ID del producto devuelva el número de unidades vendidas
DELIMITER //
CREATE OR REPLACE FUNCTION f_get_sold_units(p_product_id INT)
RETURNS INT
BEGIN
```

```
  RETURN (
    SELECT SUM(amount)
    FROM orders
    WHERE orders.product_id = p_product_id
  );
END //
DELIMITER ;

-- Función que devuelva el importe total de ventas entre dos fechas dadas
DELIMITER //
CREATE OR REPLACE FUNCTION f_purchase_between_dates(p_date1 DATE, p_date2 DATE)
RETURNS DECIMAL
BEGIN
  RETURN (
    SELECT SUM(amount * price)
    FROM orders
    NATURAL JOIN products
    WHERE (purchase_date >= p_date1 AND purchase_date <= p_date2)
  );
END //
DELIMITER ;

-- Función que, dado un año, devuelva el id del producto más vendido en ese año
DELIMITER //
CREATE OR REPLACE FUNCTION f_best_seller(p_year INT)
RETURNS INT
BEGIN
  RETURN (
    SELECT product_id
    FROM orders
    NATURAL JOIN products
    WHERE YEAR(purchase_date) = p_year
    GROUP BY product_id
    ORDER BY SUM(amount) DESC
    LIMIT 1
  );
END //
DELIMITER ;

-- Función que, dado un ID de usuario, devuelva el dinero que ha gastado en todos
sus pedidos
DELIMITER //
CREATE OR REPLACE FUNCTION f_spent_money_user(p_user_id INT)
RETURNS DOUBLE
BEGIN
  RETURN (
    SELECT SUM(amount * price)
    FROM orders
    NATURAL JOIN products
    WHERE (orders.user_id = p_user_id)
  );
END //
DELIMITER ;

-- Función que, dado un ID de producto, devuelva su stock
DELIMITER //
```

```
CREATE OR REPLACE FUNCTION f_get_stock(p_product_id INT)
RETURNS INT
BEGIN
  RETURN (
    SELECT stock
    FROM products
    WHERE (products.product_id = p_product_id)
  );
END //
DELIMITER ;

-- Procedimiento que, dado un usuario, un producto, y una cantidad,
-- cree un pedido con la fecha actual y la cantidad dada
DELIMITER //
CREATE OR REPLACE PROCEDURE p_insert_order(
  p_user_id INT,
  p_product_id INT,
  p_amount INT,
  p_purchase_date DATE
)
BEGIN
  INSERT INTO orders(user_id, product_id, amount, purchase_date)
  VALUES (p_user_id, p_product_id, IFNULL(p_amount, 1), IFNULL(p_purchase_date,
CURRENT_DATE()));
END //
DELIMITER ;

-- Procedimiento que, dado un porcentaje (ej. -0.2) modifique los precios
-- según el porcentaje dado (con -0.2, un precio de 10€ pasaría a ser de 8€)
DELIMITER //
CREATE OR REPLACE PROCEDURE p_change_prices(p_fraction DOUBLE)
BEGIN
  UPDATE products
  SET price = price + price * p_fraction;
END //
DELIMITER ;

-- Procedimiento que, dado un nombre de usuario y un año, crea una vista
-- vOrders<Nombre><Año> que contiene los pedidos realizados por el usuario en ese
año
-- Recuerde que un nombre puede contener espacios
DELIMITER //
CREATE OR REPLACE PROCEDURE p_create_orders_view(p_name VARCHAR(64), p_orders_year
INT)
BEGIN
  DECLARE v_user_id INT;
  SET v_user_id = (SELECT users.user_id FROM users WHERE users.name = p_name);
  EXECUTE IMMEDIATE CONCAT(
    'CREATE OR REPLACE VIEW vOrders', REPLACE(p_name, ' ', ''), p_orders_year, '
AS
    SELECT * FROM orders WHERE user_id = ', v_user_id, ' AND YEAR(purchase_date) =
', p_orders_year, ';'
  );
END //
DELIMITER ;
```

```
-- Procedimiento que, dado un entero N y un año, crea la vista anterior
-- para ese año y todos los usuarios que han realizado más de N pedidos
DELIMITER //
CREATE OR REPLACE PROCEDURE p_create_orders_views(p_min_orders INT, p_orders_year
INT)
BEGIN
  DECLARE v_user_name VARCHAR(64);
  DECLARE v_done BOOLEAN DEFAULT FALSE;
  DECLARE users_with_orders CURSOR FOR
    SELECT name
    FROM users
    NATURAL JOIN orders
    WHERE YEAR(purchase_date) = p_orders_year
    GROUP BY name
    HAVING COUNT(*) >= p_min_orders;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done := TRUE;

  OPEN users_with_orders;
  read_loop: LOOP
    FETCH users_with_orders INTO v_user_name;
    IF v_done THEN
      LEAVE read_loop;
    END IF;
    CALL p_create_orders_view(v_user_name, p_orders_year);
  END LOOP;
  CLOSE users_with_orders;
END //
DELIMITER ;
```

# 17. Tests

## Pedidos/tests.sql

```
--
-- Autor: Carlos Arévalo
-- Fecha creación: Noviembre de 2020
-- Fecha modificación: Noviembre 2025 (David Ruiz)
-- Descripción: Pruebas de procedimientos, funciones y disparadores para el
ejercicio de Pedidos
--

-- Inicialización de la BD con estado consistente
USE PedidosDB;

-- TABLA DE RESULTADOS DE TESTS
CREATE OR REPLACE TABLE test_results (
  test_id VARCHAR(10) NOT NULL PRIMARY KEY,
  test_name VARCHAR(200) NOT NULL,
  test_message VARCHAR(500) NOT NULL,
  test_status ENUM('PASS', 'FAIL', 'ERROR') NOT NULL,
  execution_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- PROCEDIMIENTO AUXILIAR PARA LOGGING
DELIMITER //
```

```
CREATE OR REPLACE PROCEDURE p_log_test(
  IN p_test_id VARCHAR(10),
  IN p_message VARCHAR(500),
  IN p_status ENUM('PASS', 'FAIL', 'ERROR')
)
BEGIN
  INSERT INTO test_results(test_id, test_name, test_message, test_status)
  VALUES (p_test_id, SUBSTRING_INDEX(p_message, ':', 1), p_message, p_status);
END //
DELIMITER ;

-- 1. Integridad declarativa
-- name obligatorio
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_1_1()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('1.1', 'users: name NOT NULL', 'PASS');

  CALL p_create_db();
  CALL p_populate_db();

  INSERT INTO users(name, province, start_date)
  VALUES (NULL, 'X Nuevo:name obligatorio', NOW()); -- NOK

  CALL p_log_test('1.1', 'ERROR: name NULL permitido', 'FAIL');
END //
-- startDate obligatorio
CREATE OR REPLACE PROCEDURE p_test_1_2()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('1.2', 'users: start_date NOT NULL', 'PASS');

  CALL p_create_db();
  CALL p_populate_db();

  INSERT INTO users(name, province, start_date)
  VALUES ('Nuevo', 'X Nuevo: startDate Obligatorio', NULL); -- NOK
  CALL p_log_test('1.2', 'ERROR: start_date NULL permitido', 'FAIL');
END//
-- usuario correcto
CREATE OR REPLACE PROCEDURE p_test_1_3()
BEGIN
  DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
    CALL p_log_test('1.3', 'ERROR: Fallo al insertar usuario válido', 'FAIL');

  CALL p_create_db();
  CALL p_populate_db();

  INSERT INTO users(name, province, start_date)
  VALUES ('Nuevo', '* Nuevo: Admitido', NOW()); -- OK
  CALL p_log_test('1.3', 'users: Inserción válida admitida', 'PASS');
END//
-- name único
CREATE OR REPLACE PROCEDURE p_test_1_4()
BEGIN
```

```
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('1.4', 'users: name UNIQUE', 'PASS');

  CALL p_create_db();
  CALL p_populate_db();

  INSERT INTO users(name, province, start_date)
  VALUES ('TestDup', '* Nuevo: Repetido', NOW());

  INSERT INTO users(name, province, start_date)
  VALUES ('TestDup', '* Nuevo: Repetido2', NOW());

  CALL p_log_test('1.4', 'ERROR: name duplicado permitido', 'FAIL');
END //
-- 2b. Productos --> Valores obligatorios (RN-2a);
-- Precio no puede ser nulo
CREATE OR REPLACE PROCEDURE p_test_2_1()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('2.1', 'price NOT NULL', 'PASS');

  CALL p_create_db();
  CALL p_populate_db();

  INSERT INTO products(description, price, stock)
  VALUES ('Mi Band 3', NULL, 50); -- NOK
  CALL p_log_test('2.1', 'ERROR: precio NULL permitido', 'FAIL');
END//
-- Precio >= 0
CREATE OR REPLACE PROCEDURE p_test_2_2()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('2.2', 'products: price >= 0', 'PASS');

  CALL p_create_db();
  CALL p_populate_db();

  INSERT INTO products(description, price, stock)
  VALUES ('Mi Band 3', -1, 50); -- NOK
  CALL p_log_test('2.2', 'ERROR: price negativo permitido', 'FAIL');
END//
--- Stock >= 0
CREATE OR REPLACE PROCEDURE p_test_2_3()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('2.3', 'products: stock >= 0', 'PASS');

  CALL p_create_db();
  CALL p_populate_db();

  INSERT INTO products(description, price, stock)
  VALUES ('Mi Band 3', 5, -1); -- NOK
  CALL p_log_test('2.3', 'ERROR: stock negativo permitido', 'FAIL');
END//
-- 2c. Pedidos --> Valores por defecto (RN-3a): Cant=1; purchaseDate=now()
-- valor por amount=1
```

```
CREATE OR REPLACE PROCEDURE p_test_3_1()
BEGIN
  DECLARE v_amount INT;

  DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
    CALL p_log_test('3.1', 'ERROR: Fallo al probar valor por defecto', 'FAIL');

  CALL p_create_db();
  CALL p_populate_db();

  -- Omitir amount para que use el DEFAULT
  INSERT INTO orders(user_id, product_id, purchase_date)
  VALUES (8, 2, CURDATE());

  SELECT amount INTO v_amount
  FROM orders
  WHERE order_id = LAST_INSERT_ID();

  IF v_amount = 1 THEN
    CALL p_log_test('3.1', 'orders: amount DEFAULT 1', 'PASS');
  ELSE
    CALL p_log_test('3.1', 'ERROR: amount por defecto no es 1', 'FAIL');
  END IF;
END//
-- 3. Integridad referencial (FKs): Usuario inexistente, Producto inexistentes
-- Usuario inexistente
CREATE OR REPLACE PROCEDURE p_test_4_1()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('4.1', 'FK: user_id debe existir en users', 'PASS');

  CALL p_create_db();
  CALL p_populate_db();

  INSERT INTO orders(user_id, product_id, amount)
  VALUES (99, 2, 1);
  CALL p_log_test('4.1', 'ERROR: FK user_id no validada', 'FAIL');
END //
-- Producto inexistente
CREATE OR REPLACE PROCEDURE p_test_4_2()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('4.2', 'FK: product_id debe existir en products', 'PASS');

  CALL p_create_db();
  CALL p_populate_db();

  INSERT INTO orders(user_id, product_id, amount)
  VALUES (8, 99, 1);
  CALL p_log_test('4.2', 'ERROR: FK product_id no validada', 'FAIL');
END//
-- 3b. Al borrar Usuario
CREATE OR REPLACE PROCEDURE p_test_4_3()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('4.3', 'FK: DELETE users con orders protegido', 'PASS');
```

```
  CALL p_create_db();
  CALL p_populate_db();

  DELETE FROM users WHERE user_id = 8; -- NOK

  CALL p_log_test('4.3', 'ERROR: DELETE users con orders permitido', 'FAIL');
END //
-- 3d. Al borrar Producto
CREATE OR REPLACE PROCEDURE p_test_4_4()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('4.4', 'FK: DELETE products con orders protegido', 'PASS');

  CALL p_create_db();
  CALL p_populate_db();

  DELETE FROM products WHERE product_id = 2; -- NOK

  CALL p_log_test('4.4', 'ERROR: DELETE products con orders permitido', 'FAIL');
END //
-- 4. Checks: Cant>=1 AND Cant<=10 (RN-3b)
-- amount=0
CREATE OR REPLACE PROCEDURE p_test_3_2()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('3.2', 'orders: amount >= 1', 'PASS');

  CALL p_create_db();
  CALL p_populate_db();

  INSERT INTO orders(user_id, product_id, amount)
  VALUES (8, 3, 0);
  CALL p_log_test('3.2', 'ERROR: amount = 0 permitido', 'FAIL');
END//
-- amount>10
CREATE OR REPLACE PROCEDURE p_test_3_3()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('3.3', 'orders: amount <= 10', 'PASS');

  CALL p_create_db();
  CALL p_populate_db();

  INSERT INTO orders(user_id, product_id, amount)
  VALUES (8, 3, 11);
  CALL p_log_test('3.3', 'ERROR: amount > 10 permitido', 'FAIL');
END//
-- 5. Checks: (Mes(Pedido)<>Agosto) (RN-3c)
CREATE OR REPLACE PROCEDURE p_test_3_4()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('3.4', 'orders: MONTH(purchase_date) <> 8', 'PASS');

  CALL p_create_db();
  CALL p_populate_db();
```

```
  INSERT INTO orders(user_id, product_id, amount, purchase_date)
  VALUES (8, 3, 1, '2020-08-01');
  CALL p_log_test('3.4', 'ERROR: pedido en agosto permitido', 'FAIL');
END//
-- Ejecutar todos los tests
CREATE OR REPLACE PROCEDURE p_run_all_tests()
BEGIN
  DELETE FROM test_results;

  -- Tests de Usuarios (RN-1a, RN-1b)
  CALL p_test_1_1();
  CALL p_test_1_2();
  CALL p_test_1_3();
  CALL p_test_1_4();

  -- Tests de Productos (RN-2a, RN-2b, RN-2c)
  CALL p_test_2_1();
  CALL p_test_2_2();
  CALL p_test_2_3();

  -- Tests de Pedidos (RN-3a, RN-3b, RN-3c)
  CALL p_test_3_1();
  CALL p_test_3_2();
  CALL p_test_3_3();
  CALL p_test_3_4();

  -- Tests de Integridad Referencial
  CALL p_test_4_1();
  CALL p_test_4_2();
  CALL p_test_4_3();
  CALL p_test_4_4();

  -- Mostrar resultados detallados
  SELECT test_id AS 'ID',
         test_name AS 'Test',
         test_status AS 'Estado',
         test_message AS 'Mensaje'
  FROM test_results
  ORDER BY test_id;

END//
DELIMITER ;
```